16th May : Project With RPC (Full Stack Development)

-Jaani [1]

A project has two things : Backend and Frontend.

Monorepo : A single repo that contains our backend, frontend, shared config.

Imagine a project has three things → api, web, queue etc (meaning multiple projects), so rather than storing each of them in 3 different repos, that causes us problems while maintaining the project.

In monorepo, we store everything in one repo, but the backend, frontend are isolated, so each folder has its own "package.json" etc.
But, bcs of this, code could not be shared, so to have common code, we can use Monorepo managers like:

1. Turborepo
2. NX

Turborepo is a monorepo manager, which has two special folder 'apps' and 'packages'. This is built by Vercel.

#
```
apps/
  ├→ api/
  ├→ web/      } isolated, each has own package.json
  └→ queue/
packages/  ⇒ shared code exists here
```

We can have:

api/package.json

"@chaicode-jorm/utils": "workspace:*" means

↳ use the local package from this monorepo workspace instead of downloading from npm.

⇒ This is written inside packages/utils/package.json, we see:

```
{
    "name": "@chaicode-jorms/utils"
}
```

To use this, import it:

import { xyz } from "@chaicode-jorms/utils"

RPC (Remote Procedure Calls)

Now, frontend can do remote Procedures calls, these procedures can call services, Now we need to map our frontend with these procedures.

To do so we have →

⇒ GRPC (data stored in Protobuf)

⇒ tRPC (data in JSON)

Signature of a fun is the shape/skeleton of function. Now we can serialize the signature, in binary, now we won't will send serialized data to client.

Now client only has signature, client can:

const result = client.add(5,9) → signature

Now result is a number.

⇒ Now client can send data to backend (that is the actual parameters), now can also find the procedure (corresponding). [3]

⇒ The data in binary format is called (Protobuffs), this is created by google.

tRPC (Typescript Remote Procedure Calls)

↳ Here data is stored in JSON format. It has zod, like a validation layer

↳ Here we have Type Inferencing layer

Server ⟶ Type Inferencing

↓

type t = typeof <series_procedures>

↳ Client.

Transport layer

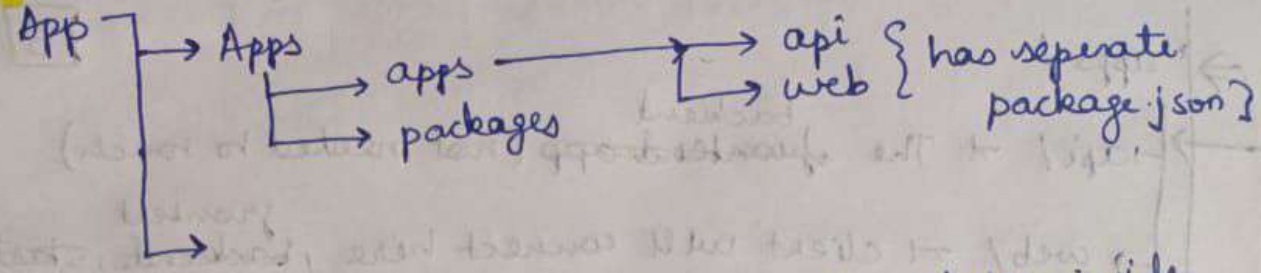
Bcs tRPC does not have a own HTTP layer, we can use express, HTTP, Fastify, Hono, as a transport layer.

Setting up the github repo.

1. Install package manager 'pnpm'
2. File structure

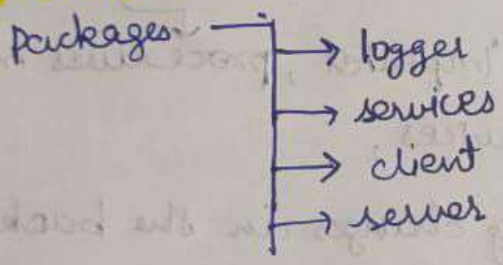
App
- → apps
- → packages
- → package.json [for the root]
- → pnpm-workspace.yaml
- ↳ setup.sh → generates symlinks

File structure



You can find symbolic link in the node modules, inside api, web, which help us find the package through symlink stored in packages. This is only during development. During deployment, the actual code is stored, not "symlink"

File structure of packages:



cors error means, the request usually does reach the server but the browser blocks the frontend from accessing the response.

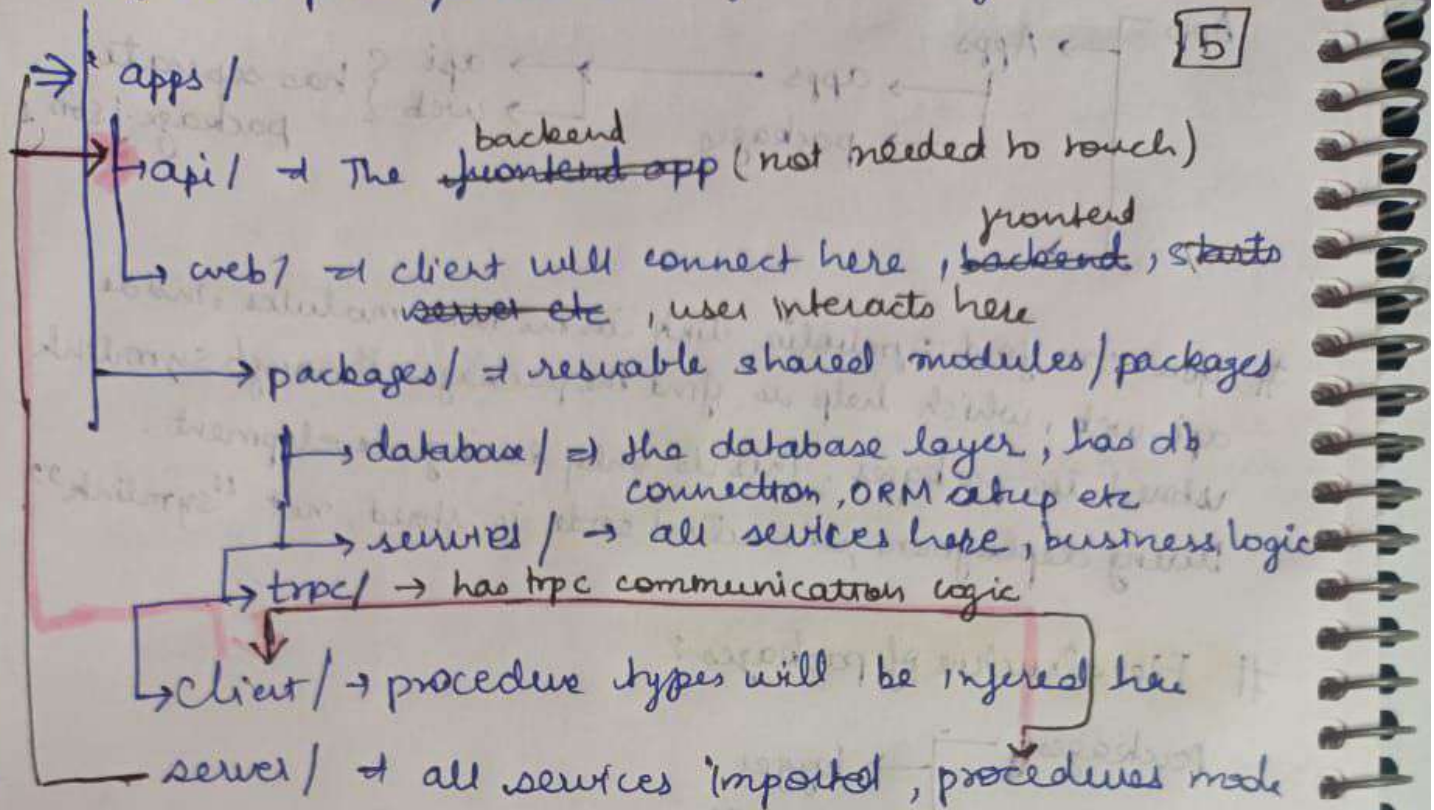
We need to write procedures in tRPC, and call them in frontend.

Rules for procedure:

1. Procedure should not contain business logic.

So our business logic will be written in services folder

gRPC expects you to write signature on your own.



⇒ All this so that if something changes in the backend, it will reflect on the frontend.

This trpc/server folder defines procedures, defines routers, and exposes backend functions to frontend.

To make '.env' common for all, we create a symlink for it, anything you change in root env, will get updated everywhere.

pnpm -dev : runs all dev commands, frontend, backend all.

Now inside services, make a 'user' services and two files 'index.ts' and 'model.ts'

~~Inside 'index.ts'~~

⇒ Try describing the fields inside the model.ts, as it helps to create good index. [6]

Try naming the service and its corresponding procedure by same name.

Here two things happens: mutation and query in the procedures.

⇒ Inside router.ts :

meta data to auto generate docs

```
export const authRouter = router({
  createEmailAndPassword : publicProcedure.meta({openapi:
    method : 'POST'
    tags : TAGS
  }).input().output().mutation(),
```

→ In the whole project:

packages / database = Model

packages / service = controller

packages / trpc = procedures

apps / web = UI bna kar procedure call karde

⇒ Now I don't want to expose my hooks, so.

earlier:

```
export fun SignUpForm({className, ...props}: React.Compon
props<"form">){
  const {mutateAsync : createUserWithGmail&PassAsync} =
    trpc.auth.createUserWithEmail&Pass.useMutation()
```